

CRISP-DM Document: AI-Powered Cyber Threat Detection using Machine Learning on Network Traffic Data

1. Business Understanding

Cybersecurity threats are increasing rapidly in both frequency and sophistication. Traditional rule-based Intrusion Detection Systems (IDS) struggle to detect novel or zero-day attacks and are prone to generating excessive false negatives. Given the growing complexity of network traffic and threat vectors, organizations need intelligent, real-time solutions.

Goal: Build an AI-powered IDS that classifies and detects various cyber threats using machine learning, providing interpretability and real-time responsiveness.

According to Cybersecurity Ventures, global cybercrime damages are expected to reach \$10.5 trillion annually by 2025, making proactive and intelligent cybersecurity systems essential for sectors like healthcare, finance, and critical infrastructure.

2. Objectives

This project aims to:

- Explore the distribution of attack types to understand their frequency and severity.
- Train supervised machine learning models to classify normal and malicious traffic.
- Explore anomaly detection techniques to detect unknown threats (zero-day attacks).
- Provide clear, interpretable insights using *LIME* for model predictions.
- Simulate real-time detection using streamlit and fastAPI.

3. Data Understanding

Dataset: UNSW-NB15

Source: Australian Centre for Cyber Security (ACCS), UNSW Canberra

Total Records: 2,540,044

- Training: 175,341 samples
- Testing: 82,332 samples

Classes:

- Normal
- Attacks: DoS, Exploits, Reconnaissance, Shellcode, Fuzzers, Analysis, Backdoors, Generic, Worms

Final Set of 42 Features and Their Definitions

Connection Info

- **proto**: Network protocol used (e.g., TCP, UDP)
- **state**: Connection state between source and destination
- **service**: Application service on the destination (e.g., http, dns, ssh)

Size & Volume

- **sbytes**: Bytes sent from source to destination
- **dbytes**: Bytes sent from destination to source
- **spkts**: Number of packets sent by source
- **dpkts**: Number of packets sent by destination
- **sload**: Source bits per second
- **dload**: Destination bits per second
- **swin**: Source TCP window size

- **dwin**: Destination TCP window size

Duration & Timing

- **dur**: Duration of connection
- **tcprtt**: TCP round-trip time (SYN-ACK + ACK time)
- **synack**: Time from SYN to SYN-ACK packet
- **ackdat**: Time from SYN-ACK to ACK
- **sinpkt**: Time between packets from source
- **dinpkt**: Time between packets from destination
- **sjit**: Jitter in source packet times
- **djit**: Jitter in destination packet times

TTL (Time to Live) & Loss

- **sttl**: TTL value from source to destination
- **dttl**: TTL value from destination to source
- **sgloss**: Source retransmissions or dropped packets
- **dloss**: Destination retransmissions or dropped packets

Statistical & Behavioral Counters

- **smean**: Average size of source packets
- **dmean**: Average size of destination packets

- `trans_depth`: Depth of HTTP request/response in session
- `response_body_len`: Size of HTTP response body
- `ct_state_ttl`: Number of state transitions by TTL ranges
- `ct_flw_http_mthd`: Number of HTTP methods like GET, POST in flow
- `is_ftp_login`: 1 if FTP login occurred, else 0
- `ct_ftp_cmd`: Number of FTP commands in session
- `ct_srv_src`: Connections with same service and source IP
- `ct_srv_dst`: Connections with same service and destination IP
- `ct_dst_ltm`: Recent connections to same destination
- `ct_src_ltm`: Recent connections from same source
- `ct_src_dport_ltm`: Recent connections from same source to same port
- `ct_dst_sport_ltm`: Recent connections to same destination from same source port
- `ct_dst_src_ltm`: Recent connections between same source and destination
- `is_sm_ips_ports`: 1 if source/destination IPs and ports match, else 0

4. Data Preparation

- Dropped Non-Generalizable Features: IPs, ports, timestamps (e.g., `saddr`, `stime`)
- Encoding: One-hot encoding for `proto`, `state`, `service`
- Scaling: `StandardScaler` applied to numerical features
- Split Strategy: Training (70%), Validation (30%)

- The dataset showed a high imbalance between normal and malicious classes. Attack types like *Exploits*, **Fuzzers, and **Reconnaissance* appeared more frequently, while *Worms* and *Backdoors* were significantly underrepresented.

5. Modeling

Unsupervised Learning (Anomaly Detection)

Model	Performance Summary
Isolation Forest	Performed moderately well but affected by high contamination (48%)
One-Class SVM	Underperformed due to rare anomaly assumption and data imbalance

Supervised Learning (Classification)

Model	Performance Summary
Logistic Regression	Solid baseline with decent recall
XGBoost (Untuned)	Strong generalization, low false negatives
XGBoost (Tuned)	Best overall model — high recall, balanced performance

Neural Network (MLP) Stable training, good recall, no overfitting observed

6. Evaluation & Interpretation

Metrics Used:

- Accuracy
- Precision
- Recall (primary metric)
- F1 Score
- Confusion Matrix

Best Model: Tuned XGBoost

- Achieved high recall on high-support classes (e.g., Class 5 and 6)
- Moderate confusion in low-support classes (e.g., Class 8, 9)
- Confusion matrix showed strong *diagonal dominance*, indicating overall good classification performance

LIME Interpretability:

Used to explain individual predictions:

- Key features influencing malicious classification: ct_state_ttl, state_CON, ct_dst_sport_ltm
- Visual explanations highlighted green bars pushing the prediction toward "malicious" and absence of red bars for opposing features in certain predictions

7. Real-Time Simulation (Planned Deployment)

Planned Deployment Tools:

- Streamlit and FastAPI for serving the model

We used streamlit to serve the User Interface where a user can upload a csv file containing network traffic logs.

The file is sent to the backend where the FastAPI handles the model logic and performs predictions on the dataset. The predictions together with the recommended response actions are added to the csv as two columns. The csv then sends the csv with predictions back to the frontend as a JSON.

The frontend receives the json dictionary and converts it into a dataframe and displays the results.

These results are also used to make visualizations on the distribution of attack categories according to count or percentages.

A summary dataframe is also generated highlighting the threats detected together with their frequency and responses to take.

All these can be downloaded from the frontend by the client.

Below are the types of attacks along with their corresponding responses:

- *Fuzzers - Immediately block the source IP*
- *Analysis - Trigger anomaly alerts on repeated or fast port scans.*
- *Backdoors - Isolate the affected host from the network.*
- *DOS - Rate-limit or temporarily blacklist attack IPs.*
- *Exploits - Log and contain affected systems.*
- *Generic - Temporarily lock user accounts after failed attempts.*
- *Reconnaissance - Redirect to honeypots or dummy services.*
- *Shellcode - Reimage or restore from a known good backup.*

8. Limitations

1. Class Imbalance: Rare classes (e.g., Worms, Shellcode) performed poorly due to limited samples.

2. Anomaly Detection Challenges: Isolation Forest and One-Class SVM struggled due to high contamination levels.

3. Limited Feature Context: Dropping IPs and ports reduced the granularity of forensic insight.

4. No Adversarial Testing: The model was not evaluated against adversarial or evasion attacks.

9. Conclusions

- Supervised ML models, especially Tuned XGBoost, outperformed unsupervised methods in detection accuracy and recall.
- LIME was used for interpreting predictions, making the system more explainable and trustworthy for cybersecurity analysts.
- Neural networks showed good learning behavior but were slightly outperformed by XGBoost in generalization.
- The system is well-prepared for deployment with strong test results and explainability, though rare attack types remain a challenge.

10. Recommendations

1. Expand Interpretability:

Integrate SHAP for both global and local explanation perspectives.

2. Complete Deployment:

Finalize real-time streaming using River, Kafka, or custom tools with .pcap integration.

3. Model Fusion:

Experiment with ensemble models combining Neural Network and XGBoost to capture complex patterns.

4. Integrate Threat Response:

Automate actions like IP blocking, alert generation, and network isolation based on prediction output.